

Home Credit Default Risk

Project: [Home Credit Default Risk](#)

Project Lead: Alex Alvarez

Team Members & Responsibilities:

- Alex Alvarez
- Jose Lucio
- Ira Vizcarra

1. Project Information

The Kaggle Home Credit Default Risk challenge focuses on building a predictive model to estimate the likelihood that a loan applicant will default. In real-world lending systems, individuals with limited or poor credit history are often unfairly rejected, so the goal of this project is to use machine learning to improve credit risk assessment and support more equitable lending decisions. The project aims to predict loan default probability, compare multiple machine learning models, enhance performance through feature engineering from financial history data, and evaluate results using the ROC-AUC metric, which measures how well predicted probabilities match actual repayment outcomes.

The overall objective is to use multiple machine learning models to help lenders better identify applicants at risk of default while also broadening access to credit for individuals with little or no credit history. By improving prediction accuracy, financial institutions can make more informed decisions that balance risk reduction with fairness. In developing this project, we also referenced prior research that demonstrated strong model performance and highlighted useful evaluation metrics such as precision and recall. Precision measures the proportion of correctly predicted defaults among all predicted positives, while recall measures the proportion of actual defaults correctly identified (Dasril et al., 2023). However, this work did not fully explain implementation details such as preprocessing and feature engineering, which motivated us to develop a more complete machine learning pipeline that integrates data processing, multiple models, and rigorous evaluation.

2. Dataset Description & Source

The 10 datasets provided by Kaggle include various features of a client's financial history and behavior. The primary dataset, `application_{train|test}.csv`, is the main table, split into two files for Train (with TARGET) and Test (without TARGET). The training set includes the target variable indicating default, while the test set does not. In addition, several tables provide deeper insights into each client's financial history. For example, `bureau.csv` and `bureau_balance.csv` describe previous loans from other financial institutions and their monthly status; `previous_application.csv` records past loan applications with Home Credit; and `installments_payments.csv` tracks repayment behavior, including missed and completed payments. Other datasets, such as `POS_CASH_balance.csv` and `credit_card_balance.csv`, provide monthly snapshots of past loan and credit card balances.

2.1 Main Dataset

- `application_train.csv`: training data containing features and target label
- `application_test.csv`: test data without target values

2.2 Supporting Datasets

Additional tables provide historical financial behavior:

- `bureau.csv`: external credit history from other institutions
- `bureau_balance.csv`: monthly bureau account status
- `previous_application.csv`: past loan applications
- `POS_CASH_balance.csv`: point-of-sale and cash loans
- `credit_card_balance.csv`: credit card usage history
- `installments_payments.csv`: repayment behavior data

These datasets allow deeper behavioral analysis beyond basic applicant information.

3. Data Preprocessing & Feature Engineering

Before testing, we set up our environment using Google Colab and their T4 GPU. We used Google Drive to store all 10 Kaggle .csv files in one large zip and used Python's `!unzip` command to import them into Colab to minimize personal computer storage usage.

3.1 Missing Value Handling:

Missing numerical values were imputed using the median strategy, which is robust to outliers and preserves distribution characteristics.

3.2 Categorical Encoding:

We performed one-hot encoding on our datasets, a process that converts categorical data into a binary format for machine learning models to read. For example, the “NAME_CONTRACT_TYPE” column had variables such as “CASH_LOANS” and “REVOLVING_LOANS”. After one-hot encoding, each variable will turn into its own column and have output variables of 0 or 1, indicating if it is true or not.

3.3 Dataset Alignment:

Train and test datasets were aligned to ensure identical feature space using:

- Column synchronization
- Missing column filling with zeros

This prevents inconsistencies during model prediction.

3.4 Feature Engineering:

Feature engineering also plays a big role in our data handling stage by transforming raw data into meaningful variables. Raw data alone is sometimes insufficient when trying to capture customer behaviors, so we use feature engineering to get informative features from previous transactions. Key features that were created include previous application information such as loan amounts and decision timing, credit card behavior including balance and payment patterns, and bureau-related metrics like credit history length, overdue payments, and total credit exposure. Installment data is also leveraged with features measuring payment behavior and ratios. Values above 1 indicate overpayment, values equal to 1 reflect full payment, and values below 1 signal potential underpayment risk. By converting raw data into more of these behavioral indicators, it allows for more accurate and robust predictive models. For each customer (SK_ID_CURR), statistical features were computed: mean, sum, maximum, minimum, standard deviation.

The engineered feature groups were:

- Bureau credit history features (overdue counts, credit sums)
- Previous application behavior (loan timing, credit amounts)
- POS CASH usage patterns
- Credit card balance behavior
- Installment repayment ratios

Key Derived Feature:

- $PAYMENT_RATIO = \frac{AMT_PAYMENT}{AMT_INSTALMENT}$
- This feature indicates repayment behavior:
 - > 1 , overpayment
 - $= 1$, full payment
 - < 1 , underpayment risk

4. Methods, Tools, and Algorithms Used

4.1 Models:

- Logistic Regression
 - A baseline linear classification model that estimates probability using a sigmoid function. We applied feature scaling (StandardScaler) due to its sensitivity to feature magnitude.
- Random Forest
 - An ensemble method that builds multiple decision trees using bootstrapped samples and averages their outputs to reduce variance and overfitting.
- Gradient Boosting
 - A sequential ensemble model where each tree learns to correct errors made by previous trees.
- Extra Trees Classifier
 - Similar to Random Forest but introduces more randomness by selecting split points randomly, improving generalization.
- XGBoost (Extreme Gradient Boosting)

- A highly optimized gradient boosting framework that improves speed and performance using regularization and parallel processing.
- HistGradientBoosting Classifier
 - A faster histogram-based gradient boosting method optimized for large datasets.
- TensorFlow Neural Network
 - A fully connected neural network was implemented using TensorFlow/Keras:

Architecture:

 - Dense layers (256 → 128 → 64 neurons)
 - Dropout layers (0.3) to reduce overfitting
 - Sigmoid output layer for probability prediction

Training:

 - Loss function: Binary Cross-Entropy
 - Optimizer: Adam
 - Evaluation metric: AUC

This model learns complex nonlinear relationships through backpropagation.

4.2 Tools:

- 4.2.1 Python (Pandas, NumPy)
- 4.2.2 Scikit-learn
- 4.2.3 XGBoost
- 4.2.4 TensorFlow / Keras
- 4.2.5 Google Colab (T4 GPU acceleration)
- 4.2.6 Matplotlib (for visualization)
- 4.2.7 Kaggle dataset ([Home Credit Default Risk](#))

4.3 Algorithms:

4.3.1 Feature Engineering Algorithm: make_features()

A custom aggregation function was developed to convert transaction-level data into customer-level features.

Steps:

1. Group data by customer ID (SK_ID_CURR)

2. Apply statistical functions:
 - a. mean, sum, max, min, standard deviation
3. Flatten multi-level columns into structured feature names
4. Merge aggregated features into main dataset

This transforms raw transactional history into meaningful behavioral signals such as:

- Credit usage patterns
- Repayment behavior
- Loan frequency

4.3.2 Evaluation Metrics & Algorithms:

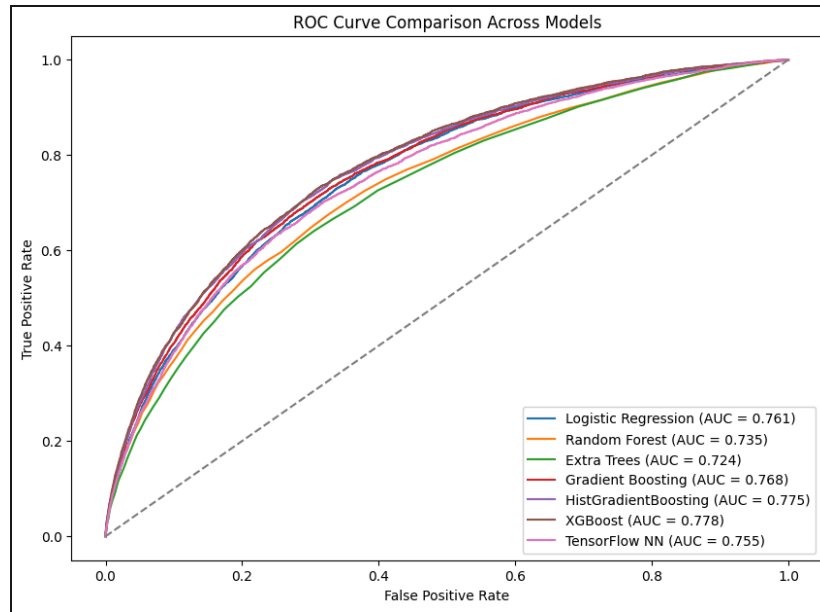
- ROC Curve:
 - Evaluates model performance across all classification thresholds by plotting:
 - True Positive Rate (Recall)
 - False Positive Rate
- AUC (Area Under Curve)
 - A single-score metric representing model quality, where a higher AUC indicates better discrimination between defaulters and non-defaulters:
 - 1.0 = perfect model
 - 0.5 = random guessing

All models were compared using ROC-AUC for fairness since the dataset is highly imbalanced.

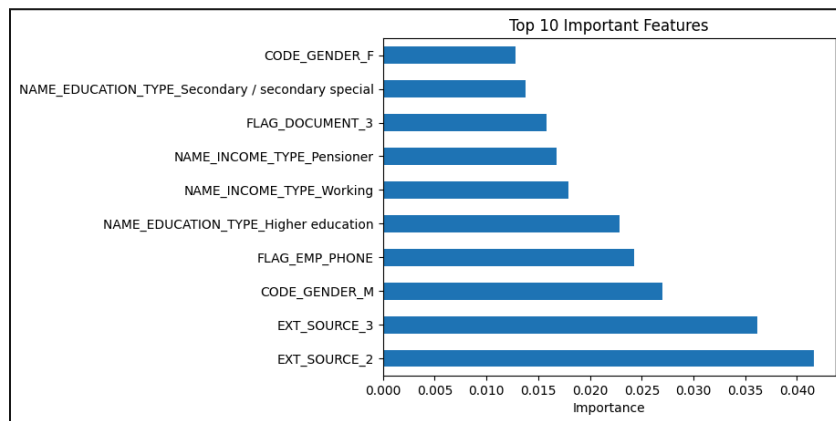
4.3.3 Preprocessing Algorithms:

- Missing Value Imputation: $x_{missing} = median(x)$
- One-Hot Encoding: Transforms categorical variables into binary vectors.
- Train-Test Alignment: Ensures identical feature structure between datasets.

5. Results



In our resulting graph, we can see that the machine learning model that performed the best was the XGBoost model, with the most optimal curve shape. The ROC curve compares True Positive Rates vs False Positive Rates, and curves that are closer to the top left of the graph indicate better performance. Additionally, when compared to our random guessing, $AUC = 0.5$ (dashed line), we can see that all of our models still performed significantly better.



We also noticed that there were many influential features that strongly impact the default risk in our prediction models. External credit risk scores, such as EXT_SOURCE_2 and EXT_SOURCE_3, are derived from third-party financial reliability assessments, and emerged as the strongest predictors of an applicant's repayment capability. Employment and income stability indicators, including FLAG_EMP_PHONE, NAME_INCOME_TYPE_Working, and

NAME_INCOME_TYPE_Pensioner, show the importance of consistent employment or recurring income in reducing the chance of defaulting. Additionally, customers with higher education levels (NAME_EDUCATION_TYPE_Higher education) are associated with lower default risk compared to customers with secondary education levels (NAME_EDUCATION_TYPE_Secondary). Lastly, demographic features such as gender (CODE_GENDER_M and CODE_GENDER_F) also show a correlation between repayment behaviors within the datasets.

After looking at our results, we saw that the XGBoost model was the best performer for predicting loan default risk. It has the ability to capture non-linear relationships, handle a large feature space, and learn complex financial patterns, while being able to effectively handle missing values. The final model was retrained on the full dataset, and our predictions were generated and exported as submission.csv, with the output format including SK_ID_CURR and the TARGET probability of default. Some of our key takeaways from the modeling process was that feature engineering had the biggest impact on model performance, and tree-based models like XGBoost performed better with linear models. XGBoost achieved the highest performance with an approximate ROC-AUC score of 0.778, demonstrating its strength as a predictive tool for assessing credit risk.

6. Conclusion

This project demonstrates the effectiveness of machine learning in predicting loan default risk using real-world financial data. Through feature engineering and multiple model comparisons, we achieved strong predictive performance.

XGBoost was identified as the best-performing model, making it suitable for deployment in risk assessment systems.

6.1 Limitations

- 6.1.1 Computational cost due to large dataset size sometimes led to the RAM crashing and thus the entire .ipynb having to restart, costing a copious amount of time.
- 6.1.2 Limited hyperparameter tuning.
- 6.1.3 Class imbalance still affects recall performance.

6.2 Future Improvements

- 6.2.1 Advanced hyperparameter optimization (can use GridSearch or Optuna)
- 6.2.2 Stacking ensemble models (crashed, refer to 6.1.1)
- 6.2.3 Feature selection techniques to reduce dimensionality

7. Sources

1. Dasril, Y., Arisandy, Y., & Salahudin, S. N. (2023). Home credit default risk assessment using embedded feature selection and stacking ensemble technique. *Journal of Numerical Optimization and Technology Management*, 1(2), 59–68.
2. Baladram, S. (2024, November 17). Extra Trees, explained: A visual guide with code examples. Medium.
<https://medium.com/@samybaladram/extra-trees-explained-a-visual-guide-with-code-examples-4c2967cedc75>
3. GeeksforGeeks. (2026, May 2). Random forest algorithm in machine learning.
<https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>
4. Clark, B., & Lee, F. (n.d.). What is gradient boosting? IBM.
<https://www.ibm.com/think/topics/gradient-boosting>
5. GeeksforGeeks. (2025, July 23). HistGradientBoostingClassifier in sklearn.
<https://www.geeksforgeeks.org/machine-learning/histgradientboostingclassifier-in-sklearn/>
6. NVIDIA. (n.d.). What is XGBoost and why does it matter? NVIDIA Glossary.
<https://www.nvidia.com/en-us/glossary/xgboost/>